

Lab 10 Template

Part 01

1. **“outfile-Good text” pdf and “out-file Bad text” pdf (upload the files in canvas submission)**
(5 points)
2. **Screenshot of the hashes of the text PDFs colliding for SHA1 and differing with SHA512**
(5 points)

```
cpre3310@cpre3310:~/lab10/part01/sha1collider$ sha1sum out-*.pdf
23c5fcc955362827f829eccb041776dc6164631c  out-bad.pdf
23c5fcc955362827f829eccb041776dc6164631c  out-good.pdf
cpre3310@cpre3310:~/lab10/part01/sha1collider$ sha512sum out-*.pdf
b7901e81d0e7795dcf7d52c9c0fdedb335008e8ea4eceb0540f7fe9627162956a6d8097fd1c1cab71a75c7a980622f9a60e30ef04a4ba6f3810b0faf5edb907f  out-bad.pdf
427ed2c84f1517239811dd0f8fea8fee3b1b1da61fbb23ff6418369f01a9e0740774f438b26ede2c78b824c05d5f0af7c739499575df47f012d2374adce32797  out-good.pdf
```

3. **Explain why the SHA1 hashes are the same for different files, but the SHA512 hashes are different. Please be specific about how the near-collision/collision works.**
(5 points)

The SHA-1 hashes are basically same because a collision was intentionally generated using a collision attack tool, which basically modifies both PDF files by inserting specially crafted data blocks that basically do not affect how the files appear visually but manipulate the internal hash computation so that both files basically produce the same SHA-1 hash, which works because SHA-1 is vulnerable to collision attacks because of its weaker 160-bit design.

SHA-512 produces different hashes because it uses a stronger algorithm with a larger 512-bit output and an improved internal structure, making it resistant to known collision attacks. The inserted collision blocks do not result in the same hash under SHA-512, so that's why the output differs.

4. **How likely is this to be used to carry out an attack in the wild? Give an example/support your claim**

(10 total points, 5 points likelihood, 5 points example)

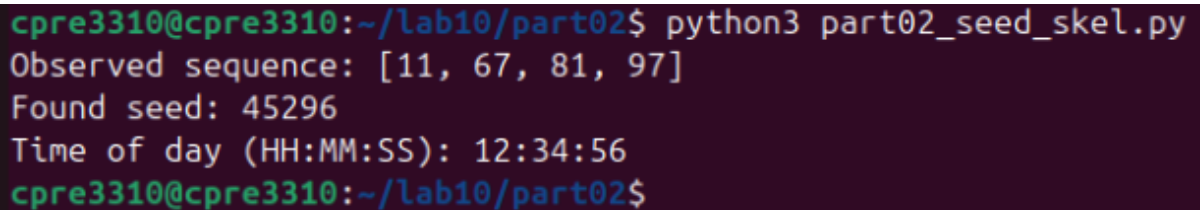
SHA-1 should no longer be used because it is vulnerable to collision attacks an attacker could create 2 different files with the same hash, breaking the integrity guarantee of hashing.

For example, an attacker could create a malicious file that has same SHA-1 hash as trusted document, which could allow the attacker to trick systems into accepting then malicious files as legitimate ones, however, this attack is difficult and requires significant computational resources and because of that it is not so commonly used in everyday attacks and despite this, SHA-1 is considered insecure and has been replaced by stronger algorithms like SHA-256 and SHA-512.

Part 02

5. **Take a screenshot of your output with both values (found seed and time of day) clearly displayed and include it in your lab report.**

(5 points)



```
cp3310@cp3310:~/lab10/part02$ python3 part02_seed_skel.py
Observed sequence: [11, 67, 81, 97]
Found seed: 45296
Time of day (HH:MM:SS): 12:34:56
cp3310@cp3310:~/lab10/part02$
```

6. **Upload your code to Canvas.**

(5 points)

7. **Once you recover the seed, can you predict only future random numbers, or can you also reconstruct the past? (forward and backward predictability) Explain your reasoning.**

(10 total points, 5 points answer, 5 points explanation)

Yes, we can predict both future and past numbers if the seed is known to us. The Mersenne Twister is basically a deterministic pseudorandom number generator, which means that the sequence of numbers it produces is basically determined by the initial seed. By reinitializing the generator with the same seed, the exact same sequence can be reproduced, which allows the prediction of future values and reconstruction of past values, which makes it unsuitable for cryptographic purposes.

Part 03

8. Include the 5 different screenshots in your lab report.
(5 points)

```
cpre3310@cpre3310:~/lab10/part03$ python3 part03_compare_prng_csprng_skel.py
PRNG values (random.randint): [213, 5, 152, 188, 99, 138, 223, 82]
CSPRNG values (secrets.randbelow): [120, 169, 125, 62, 46, 215, 1, 118]
CSPRNG values (os.urandom):      ['0x10', '0x1d', '0x91', '0x2', '0x35', '0xb1', '0xcf', '0xc8']
Raw os.urandom bytes (hex):      101d910235b1cfc8
```

Rerun this script several times: I

Record the output by taking a screenshot of each run.

```
cpre3310@cpre3310:~/lab10/part03$ python3 part03_compare_prng_csprng_skel.py
PRNG values (random.randint): [213, 5, 152, 188, 99, 138, 223, 82]
CSPRNG values (secrets.randbelow): [167, 168, 175, 123, 220, 101, 111, 95]
CSPRNG values (os.urandom):      ['0xe7', '0x41', '0xf', '0x7b', '0x48', '0x3f', '0x42', '0xb0']
Raw os.urandom bytes (hex):      e7410f7b483f42b0
```

Rerun this script several times:

Record the output by taking a screenshot of each run.

```
cpre3310@cpre3310:~/lab10/part03$ python3 part03_compare_prng_csprng_skel.py
PRNG values (random.randint): [213, 5, 152, 188, 99, 138, 223, 82]
CSPRNG values (secrets.randbelow): [137, 53, 41, 109, 226, 64, 130, 96]
CSPRNG values (os.urandom):      ['0x0', '0xc1', '0x9c', '0x34', '0x41', '0x24', '0x54', '0xce']
Raw os.urandom bytes (hex):      00c19c34412454ce
```

Rerun this script several times:

Record the output by taking a screenshot of each run.

```
cpre3310@cpre3310:~/lab10/part03$ python3 part03_compare_prng_csprng_skel.py
PRNG values (random.randint): [213, 5, 152, 188, 99, 138, 223, 82]
CSPRNG values (secrets.randbelow): [131, 222, 186, 225, 185, 156, 0, 166]
CSPRNG values (os.urandom):      ['0xe', '0x11', '0xbd', '0x17', '0x4a', '0xef', '0x9f', '0xfe']
Raw os.urandom bytes (hex):      0e11bd174aef9ffe
```

Rerun this script several times:

Record the output by taking a screenshot of each run.

```
cpre3310@cpre3310:~/lab10/part03$ python3 part03_compare_prng_csprng_skel.py
PRNG values (random.randint): [213, 5, 152, 188, 99, 138, 223, 82]
CSPRNG values (secrets.randbelow): [196, 183, 126, 165, 72, 206, 145, 165]
CSPRNG values (os.urandom):      ['0xb7', '0x27', '0x93', '0xa6', '0xaf', '0xb5', '0x6d', '0x5a']
Raw os.urandom bytes (hex):      b72793a6afb56d5a
```

Rerun this script several times:

Record the output by taking a screenshot of each run.

9. Compare the output of the three functions. Comment on the values being produced in each of the 5 iterations and whether each should be used in a cryptographic key.

Over here, basically across the 5 iterations, the values from `random.randint()` is followed by the same pattern every time and the PRNG output stayed same across all the executions, which shows that it is deterministic and this means the numbers only appear random, but they can be reproduced if the seed is known and because of that `random.randint()` should not be used to generate a cryptographic key and the document

also states that python's random module is based on the Mersenne Twister and is not cryptographically secure.

The values from `secrets.randbelow()` were basically different in each of the 5 iterations, which shows that the generator is using a stronger source of randomness and does not repeat a predictable sequence like the PRNG, and since the secrets module is designed for the security-sensitive randomness, it should be used for cryptographic keys, and the lab document also basically identifies secrets as using the operating system's cryptographically secure source.

The values from `os.random()` also changed in every run, and they were basically shown as raw random bytes in hexadecimal form and these bytes looked highly unpredictable and did not repeat in my 5 iterations and since `os.urandom()` gets randomness from the operating system's secure entropy source, it should also be used for cryptographic keys and our lab document also lists `os.random` as one of the cryptographically secure generator.

10. Explain what the output from 1000 iterations of the generator means and why it is or isn't useful in a cryptographic application. Include a screenshot of your final output in your lab report.

(5 points)

The output basically shows how many unique values are generated basically after 1000 iterations and a higher number of unique values also indicates better randomness and less repetition. Also, while a large number of unique values suggests good distribution, it actually does not guarantee cryptographic security, so a generator like Mersenne Twister will still produce many unique values but still remain predictable if the seed is known and therefore, this metric alone is not sufficient to determine whether a generator is basically secure for cryptographic applications or not.

11. If an attacker could see some of the values from secrets and os.urandom, could they guess future values or recreate the internal state of either generator? Provide your reasoning.

(10 total points, 5 points answer, 5 points reasoning)

The attackers cannot basically predict the future values or reconstruct the internal state of secrets or the `os.urandom` as these generators use entropy from the operating system and are basically designed to be cryptographically secure, so even if some outputs are observed, it is computationally infeasible for an attacker to determine the future values or reverse-engineer the internal state.

Part 04

12. Screenshot of the output to include in your lab report.
(5 points)

```
cpre3310@cpre3310:~/lab10/part04$ python3 part04_csprng_keys_skel.py
Generating 5 random AES keys and IVs using CSPRNGs:

--- Set 1 ---
AES Key (128-bit): 21897f5f561c06d6bc7d667f1f1fdca2
IV (128-bit):      3f0bf77ea0bde7d0f7988d8455a1ccdb
SHA-256(Key):      ae2c3954c713d020766bf8f8e4a37850fe409b955335592fd17a1d59d215d24e4

--- Set 2 ---
AES Key (128-bit): b9cd161f6e0cbdaab113e68e168f666b
IV (128-bit):      8543890f480327f59b7474704079a6e5
SHA-256(Key):      0966bc3b98c7e360c34bd7e1be80dc148cce2d5370d50ec8638b83f60c340e75

--- Set 3 ---
AES Key (128-bit): 630ac04df2bbc5edf63140108cbc4fee
IV (128-bit):      48f44fff7975b479e22dfc82c7874a4b
SHA-256(Key):      d461b7ec62eae8981d8128b612ad8b42824e3f65ce23bc64e054e18186c3d830

--- Set 4 ---
AES Key (128-bit): d97ef736762a0d95308cd2ec0ea114e2
IV (128-bit):      7ede850e6b600a70408d621c4c874d11
SHA-256(Key):      b226de8f59ef72c37e7dc8bfea20ae1918dada0faf5d30aa7a1545b82abed81a

--- Set 5 ---
AES Key (128-bit): ae27eadac232fb7ee573c97a20176f9d
IV (128-bit):      363c2c16242a0477e6dc8137ac73b0ae
SHA-256(Key):      6657b1e67cc9ea75ec24fd6e4d793e63c0171202508dda995dc12d7aef343bb6

cpre3310@cpre3310:~/lab10/part04$
```

13. Do any of the 5 generated keys and IVs repeat? What could happen in AES encryption if the same IV were used with the same key?
(10 total points, 5 points answer, 5 points reasoning)

Over here, basically none of the generated keys or IVs repeat because they are generated using a cryptographically secure random generator.

Over here, if the same IV is basically reused with the same key in AES encryption, then it could lead to serious security vulnerabilities and also, reusing an IV can cause patterns in the plaintext to appear in the ciphertext, which may basically allow an attacker to gain information about the original data, which will weaken the security of the encryption and could potentially lead to attacks such as plaintext recovery.

14. Do the hashes differ a lot between the different keys? How much would the hashes differ if the keys only differed slightly (like by one bit)? What does this ensure?

(10 total points, 5 points answer, 5 points reasoning)

Over here, the SHA-256 hashes of the keys differ significantly even though the keys are generated in a similar way, which basically demonstrates the avalanche effect, where a small change in input will produce a completely different hash output.

If 2 keys, basically differ by 1 bit, then also their hashes would appear entirely different, which will basically ensure that hash functions provide strong data integrity and make it infeasible to predict relationships between the inputs and outputs.